MICROSERVICES MINI PROJECT

VEHICLE RENTAL SYSTEM

PROJECT REPORT

Particular	Details
Course Name	Modern Cloud Service Architectures
Course Code	UQ25CA754AD3
Programme / Semester	MCA / III Semester
Team No.	13
Submitted to	Dr. C. Menaka
Department	Department of Computer Applications
University	PES University
Academic Year	2026–2027

Submitted by :-

Sl. No.	Student Name	SRN	Primary Responsibility
1	Md Imtiyaz Alam	PES1PG25CA124	User Service
2	Momin Mukhtarali	PES1PG25CA130	Vehicle Service
3	Mritunjai Kumar	PES1PG25CA134	Booking & Service Communication
4	Mohammed Hasan Musamji	PES1PG25CA129	Payment, Circuit Breaker & Saga
5	Mounish B	PES1PG25CA133	API Gateway, Integration & Testing

Table of Contents

1. Abstract

2. Problem Statement and Objectives

3. Requirements Analysis

4. Microservices Identification and Responsibilities

5. System Architecture

6. Technology Stack

7. RESTful API Design

8. API Versioning

9. Inter-Service Communication

10. Service Discovery

11. Database per Service

12. Data Isolation

13. API Gateway

14. Circuit Breaker and Fault Tolerance

15. Saga Pattern and Distributed Transaction

16. Messaging System (If Implemented)

17. API Testing and Results

18. Failure Scenarios and Resilience Testing

19. Implementation Screenshots

20. Challenges Faced and Solutions

21. Team Contribution

22. Conclusion and Future Enhancement

23. References

24. Appendix

1. Abstract

The Vehicle Rental System is a small-scale distributed application developed to demonstrate the core principles of microservices architecture. The system separates user management, vehicle management, booking coordination and mock payment processing into independent Flask services. An API Gateway provides a single entry point for client requests, while a Service Registry stores service names and their corresponding base URLs. Each major business service owns a separate SQLite database, supporting data ownership and isolation.

The Booking Service coordinates the rental workflow by retrieving user and vehicle information through REST APIs, checking vehicle availability, reserving the vehicle and requesting payment processing. The Payment Service simulates successful and failed outcomes without connecting to a real payment provider. If payment fails after a vehicle has been reserved, the Saga compensation flow releases the vehicle and marks the booking as failed or cancelled, according to the implemented response model. A Circuit Breaker is included around the payment dependency to demonstrate resilience when repeated failures occur.

The implementation uses Python, Flask, SQLite, the Requests HTTP client and Postman for API testing. Versioned endpoints under /api/v1 and /api/v2 demonstrate API evolution. The project focuses on service decomposition, RESTful design, service discovery, inter-service communication, database-per-service, gateway routing, fault tolerance and distributed transaction compensation. The implementation is evaluated using Postman requests and recorded test results covering service discovery, API routing, booking workflows and payment-failure compensation.

2. Problem Statement and Objectives

2.1 Problem Statement

A vehicle rental workflow requires user details, vehicle availability, booking records and payment outcomes to be coordinated reliably. In a tightly coupled monolithic application, changes or failures in one component can affect the entire application, and shared database access can make service boundaries difficult to maintain. The project addresses this by

separating the main business capabilities into independent services that communicate through REST APIs.

2.2 Objectives

- Develop independent User, Vehicle, Booking and Payment services.
- Expose REST endpoints using appropriate HTTP methods, including GET, POST, PUT and DELETE where supported.
- Implement API versioning with /api/v1 and /api/v2 routes.
- Register and discover services through a Service Registry.
- Use an API Gateway as the client-facing entry point.
- Keep a separate SQLite database for each major business service.
- Demonstrate inter-service communication across multiple services.
- Implement a Circuit Breaker around the payment dependency and handle failures gracefully.
- Implement a Saga-style booking workflow with a compensating vehicle-release action after payment failure.
- Test API behaviour using Postman and document evidence from the actual running system.

3. Requirements Analysis

3.1 Functional Requirements

- Create, retrieve, update and delete user records through the User Service.
- Create, retrieve and update vehicle records and manage vehicle availability through the Vehicle Service.
- Create and retrieve bookings through the Booking Service.
- Retrieve user and vehicle information through their APIs before proceeding with a booking.
- Reserve an available vehicle, request a mock payment and confirm the booking when payment succeeds.
- On payment failure, attempt the compensating action to release the reserved vehicle and cancel/fail the booking.
- Route supported API requests through the API Gateway and provide versioned vehicle endpoints.
- Register service URLs and allow services or the gateway to discover services by name.

3.2 Non-Functional Requirements

- Modularity: each service has a focused responsibility and can be run independently.
- Data isolation: each service owns its database; other services access data through APIs.
- Resilience: timeouts, error responses and Circuit Breaker behaviour help handle dependency failures.
- Maintainability: endpoints and responsibilities are separated by service and API version.
- Testability: endpoints can be tested independently and through the Gateway using Postman.
- Security and privacy: the prototype uses mock payment outcomes and should not contain real payment credentials or sensitive personal data.

3.3 Intended Actors

- Client/API tester: invokes endpoints through Postman or the Gateway.
- Rental user: represented by a user record used in booking requests.
- Rental operator/system: manages vehicle records and availability.
- Internal services: communicate to validate users, reserve vehicles and process mock payments.

4. Microservices Identification and Responsibilities

Component	Port	Responsibility	Database
User Service	5001	User CRUD operations and user record retrieval.	user.db
Vehicle Service	5002	Vehicle catalogue, vehicle status, reserve and release operations.	vehicle.db
Booking Service	5003	Coordinates booking validation, reservation, payment call and booking status.	booking.db
Payment Service	5004	Processes mock payment requests and stores payment outcomes.	payment.db
API Gateway	5000	Single client entry point; forwards supported API requests to the target service.	None
Service Registry	5005	Stores service-name to base-URL registrations for service discovery.	In-memory registry

4.1 Why Microservices Instead of a Monolith?

The service boundaries align with distinct business capabilities. Each business service owns its own database and API, so the Booking Service does not directly query the User or Vehicle database. This separation supports independent development and testing and makes failure boundaries easier to demonstrate. The prototype is intentionally small and does not claim production-scale deployment or automatic horizontal scaling.

5. System Architecture

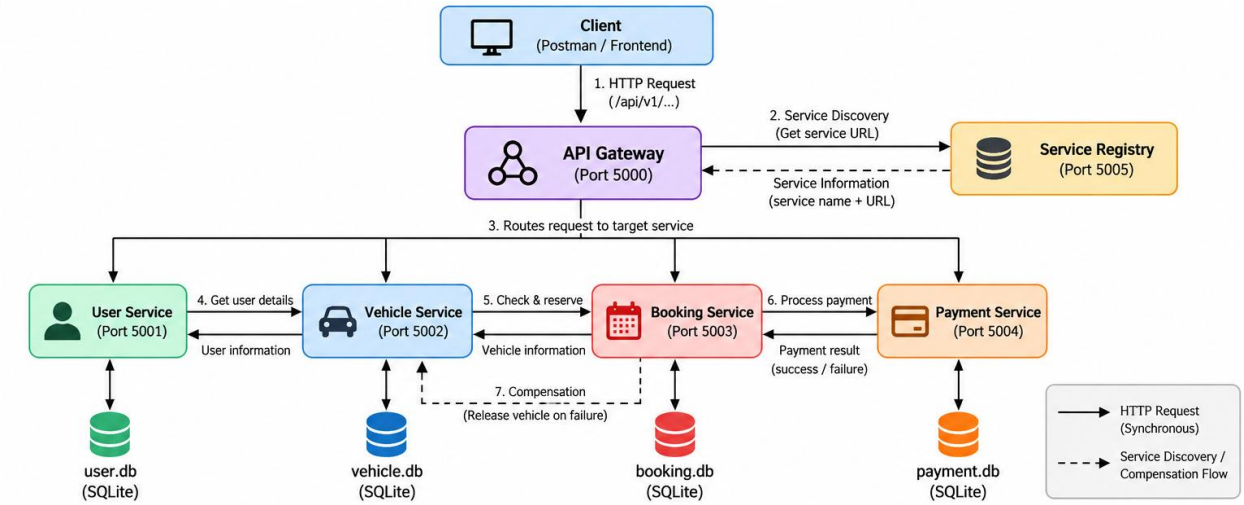


Figure 1: System Architecture of the Vehicle Rental System

Complete Request Flow

1. The client sends a request to the API Gateway or directly to a service for service-specific testing.
2. The Gateway resolves the requested service through the registry and forwards the request.
3. For a booking, Booking Service looks up User Service and Vehicle Service URLs through the registry.
4. Booking Service retrieves the user and vehicle details over HTTP and verifies that the vehicle is available.
5. Booking Service reserves the vehicle and records a pending booking.
6. Booking Service calls Payment Service through the Circuit Breaker-protected dependency.
7. If payment succeeds, the booking is marked confirmed. If payment fails, the Saga compensation attempts to release the vehicle and marks the booking as failed/cancelled.

6. Technology Stack

Technology	Usage
Python	Implementation language for service logic.
Flask	HTTP REST API framework for each service.
SQLite	Independent local database for User, Vehicle, Booking and Payment services.
Requests	Synchronous HTTP communication between services and registry.
Postman	Manual and collection-based API testing.
Git / GitHub	Version control and project source-code management using Git and GitHub.
VS Code / Terminal	Development and local service execution environment.

The project does not implement a real payment gateway. Payment status is simulated through request data for demonstration and testing.

7. RESTful API Design

The following tables describe the primary REST API endpoints implemented in the Vehicle Rental System, including their HTTP methods, purposes, input parameters and expected responses. The endpoints support user management, vehicle management, booking coordination, mock payment processing and service discovery.

7.1 User Service (port 5001)

Method	Endpoint	Purpose	Input	Expected result
GET	/api/v1/users	List users	None	JSON array of users; 200
GET	/api/v1/users/{id}	Retrieve one user	Path ID	User object; 200 or 404
POST	/api/v1/users	Create user	name, email, phone	Created user; normally 201
PUT	/api/v1/users/{id}	Update user	User fields	Updated user or error; 200/404
DELETE	/api/v1/users/{id}	Delete user	Path ID	Confirmation or error; 200/404

7.2 Vehicle Service (port 5002)

Method	Endpoint	Purpose	Input	Expected result
GET	/api/v1/vehicles	List vehicles	None	JSON array; 200
GET	/api/v1/vehicles/{id}	Retrieve vehicle	Path ID	Vehicle object; 200 or 404
POST	/api/v1/vehicles	Create vehicle	name, vehicle_number, vehicle_type, price_per_day	Created vehicle; normally 201
PUT	/api/v1/vehicles/{id}	Update vehicle	Vehicle fields/status	Updated vehicle; 200/404
DELETE	/api/v1/vehicles/{id}	Delete vehicle	Path ID	Confirmation or error
PUT	/api/v1/vehicles/{id}/reserve	Reserve vehicle	Path ID	Updated status
PUT	/api/v1/vehicles/{id}/release	Release vehicle	Path ID	Updated status

7.3 Booking Service (port 5003)

Method	Endpoint	Purpose	Input	Expected result
GET	/api/v1/bookings	List bookings	None	JSON array; 200
GET	/api/v1/bookings/{id}	Retrieve booking	Path ID	Booking object or 404
POST	/api/v1/bookings	Start booking workflow	user_id, vehicle_id, dates, payment_status	Confirmed booking or failure/compensation result
GET	/api/v1/bookings/circuit-breaker	Inspect breaker state	None	State and failure count

7.4 Payment Service (port 5004)

Method	Endpoint	Purpose	Input	Expected result
GET	/api/v1/payments	List payments	None	JSON array; 200
GET	/api/v1/payments/{id}	Retrieve payment	Path ID	Payment object or 404
POST	/api/v1/payments	Simulate payment	booking_id, user_id, amount, payment_status	Stored payment result; success or failure response

7.5 Registry and Gateway

Component	Method	Endpoint	Purpose
Registry	GET	/health	Registry health check
Registry	POST	/register	Register service name and URL
Registry	GET	/services	List registered services
Registry	GET	/services/{service_name}	Discover one service
Gateway	GET	/health	Gateway health check
Gateway	GET/POST/PUT/DELETE	/api/v1/{service}/...	Forward supported V1 routes
Gateway	GET	/api/v2/vehicles/...	Forward V2 vehicle routes

8. API Versioning

The project uses URI-based versioning. Version 1 endpoints are exposed under /api/v1, while selected Vehicle Service endpoints are available under /api/v2. This makes it possible to evolve a response format without immediately replacing the original endpoint.

Version	Example	Role
V1	GET /api/v1/vehicles	Original vehicle list/detail representation.
V2	GET /api/v2/vehicles	Versioned vehicle representation with an api_version marker and additional summary-friendly fields.
V2 summary	GET /api/v2/vehicles/{id}/summary	Returns a compact summary for one vehicle, according to the implemented V2 handler.

The Postman screenshots in Section 19 demonstrate the V1 and V2 vehicle endpoints through the API Gateway. The V2 response includes an api_version field and a vehicle count, illustrating the versioned response format.

9. Inter-Service Communication

The services communicate using synchronous HTTP requests through the Requests library. During booking creation, Booking Service obtains User Service and Vehicle Service URLs from the registry, retrieves the requested user and vehicle, asks Vehicle Service to reserve the vehicle, and calls Payment Service to process the mock payment.

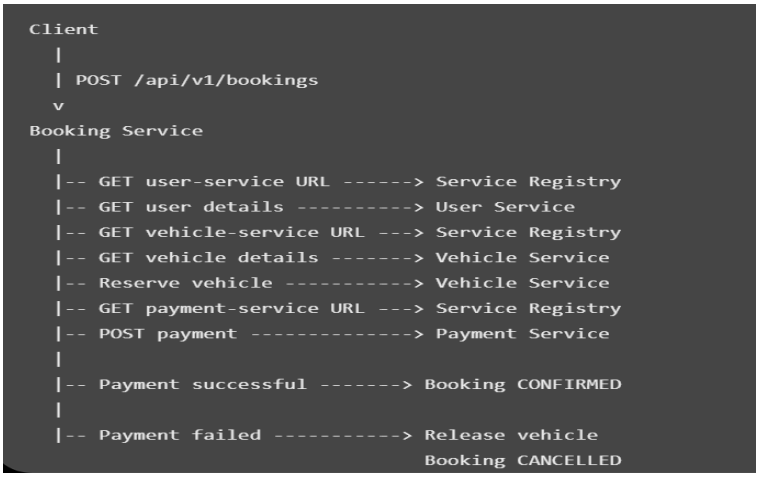


Figure 2. Logical Booking Sequence Across Microservices

10. Service Discovery

The Service Registry runs on port 5005 and stores service names and base URLs. Each service registers itself at startup using a POST request to /register. Other services request /services/{service_name} when they need to locate a dependency. The registry used in this prototype is in-memory, so registrations must be recreated after the registry restarts.

```
POST-http://localhost:5005/register
{
  "service_name":"vehicle-service",
  "service_url":"http://localhost:5002"
}

GET http://localhost:5005/services/vehicle-service
```

The Service Registry returns the registered service name and its base URL when a discovery request is made. The registry listing can be verified using the Postman evidence in Section 19.

11. Database per Service

Database	Owner	Important data (conceptual)	Purpose
user.db	User Service	id, name, email, phone	Stores user profiles used in rental requests.
vehicle.db	Vehicle Service	id, name, vehicle_number, vehicle_type, price_per_day, status	Stores vehicle details and availability.
booking.db	Booking Service	booking ID, user ID, vehicle ID, dates, status, amount/timestamps as implemented	Stores booking workflow records.
payment.db	Payment Service	payment ID, booking ID, user ID, amount, payment status, created time	Stores mock payment outcomes.

Each business service maintains its own SQLite database for the data it manages. This database-per-service approach separates data ownership and allows other services to access required information through REST APIs rather than directly querying another service's database.

12. Data Isolation

Each business service opens and queries only its own SQLite database. For example, Booking Service does not execute SQL against user.db or vehicle.db. It retrieves user details by calling User Service and obtains vehicle details or changes vehicle status by calling Vehicle Service.

Payment information is similarly handled by Payment Service APIs. This boundary prevents services from becoming directly dependent on another service's storage schema.

- User Service owns user.db.
- Vehicle Service owns vehicle.db.
- Booking Service owns booking.db.
- Payment Service owns payment.db.
- Cross-service data is exchanged through HTTP API requests, not direct cross-database queries.

13. API Gateway

The API Gateway runs on port 5000 and acts as the single client-facing entry point for supported routes. It maps service path names such as users, vehicles, bookings and payments to registry service names, resolves the target URL and forwards the HTTP method, request path, query parameters and request body to the selected service. It also exposes V2 vehicle routes.

Gateway route	Target service	Purpose
/api/v1/users/...	User Service	User API forwarding
/api/v1/vehicles/...	Vehicle Service	Vehicle API forwarding
/api/v1/bookings/...	Booking Service	Booking API forwarding
/api/v1/payments/...	Payment Service	Payment API forwarding
/api/v2/vehicles/...	Vehicle Service	V2 vehicle API forwarding

The API Gateway's request-forwarding behaviour is demonstrated by the successful vehicle-list response captured in Section 19.

14. Circuit Breaker and Fault Tolerance

The Booking Service uses a Circuit Breaker around the Payment Service call. The implementation tracks failure count and state. In CLOSED state, calls are allowed. After the configured failure threshold is reached, the breaker transitions to OPEN and temporarily rejects further dependency calls. After the recovery timeout, it permits a trial call in HALF-OPEN. A successful call resets the failure count and closes the breaker; a failed call records another failure.

State	Meaning	Expected demonstration
CLOSED	Payment dependency calls are allowed.	Inspect status before repeated failures.
OPEN	Calls are temporarily blocked after reaching the failure threshold.	Trigger repeated failures and inspect the state endpoint.
HALF-OPEN	A trial call is allowed after the recovery timeout.	Wait for the timeout and observe the next attempt.
CLOSED again	A successful trial resets the breaker.	Show successful request and reset state.

The captured Postman evidence confirms that the Circuit Breaker status endpoint returns the initial CLOSED state, with a failure count of zero and a configured failure threshold of three. The OPEN and HALF-OPEN transitions require additional testing and are not considered verified by the initial status response alone.

15. Saga Pattern and Distributed Transaction

The booking workflow is a multi-service transaction involving Booking Service, Vehicle Service and Payment Service, with User Service used for validation. The steps are not one database transaction because each service owns its database. Instead, the Booking Service coordinates the sequence and invokes a compensating action if a later step fails.

```
SUCCESS                                                                 PATH
1.           Validate           user           through           User           Service
2.          Validate          vehicle          through          Vehicle          Service
3.                                  Reserve                                    vehicle
4.               Process               mock               payment               (SUCCESS)
5.                       Mark                          booking                          CONFIRMED

FAILURE                                                                 PATH
1.                Validate               user                and                vehicle
2.                                  Reserve                                    vehicle
3.      Process      mock      payment      (FAILED      or      communication      error)
4.             Invoke             compensation:             release             vehicle
5. Mark booking FAILED/CANCELLED
```

The compensating action is the Vehicle Service release endpoint. The compensation itself can fail if Vehicle Service is unavailable; such an outcome must be reported and should be included in resilience testing rather than treated as a guaranteed rollback.

The Saga compensation workflow is demonstrated by the successful booking and failed-payment Postman responses in Section 19. Following payment failure, the Booking Service attempts to release the reserved vehicle. The subsequent vehicle response confirms that the vehicle returned to AVAILABLE status.

16. Messaging System (If Implemented)

No message broker, queue or topic is included in the current described implementation. Service-to-service communication is synchronous HTTP through Requests. Therefore, producer/consumer and asynchronous messaging details are not applicable to this version. A future enhancement could introduce a broker such as RabbitMQ or Kafka if asynchronous events become a project requirement.

17. API Testing and Results

Postman was used to test the Service Registry, business services, API Gateway, versioned endpoints and booking workflows. The table summarizes the observed results. Tests marked Partial require further verification before their corresponding functionality can be considered fully tested.

Test ID	Feature / Scenario	Expected result	Actual result	Status code	Pass/Fail
T01	Registry health	Registry returns healthy status	Registry returned service-registry with status UP.	200 OK	Pass
T02	Service registration/discovery	Registered service name returns base URL	Four services were returned: booking, payment, user and vehicle services, with their URLs.	200 OK	Pass
T03	User CRUD	Create/read/update/delete behaves as expected	Get All Users returned user records successfully. Create, update and delete require separate verification.	200 OK	Partial
T04	Vehicle CRUD	Create/read/update and status operations work	Get All Vehicles returned vehicle records. Vehicle release and status change were also verified.	200 OK	Partial
T05	Gateway GET	Gateway returns target service response	Gateway successfully returned the vehicle list.	200 OK	Pass
T06	V1/V2 versioning	Both supported vehicle API versions respond	V1 returned the vehicle list; V2 returned api_version: v2 and count: 3.	200 OK	Pass
T07	Successful booking	Available vehicle reserved; payment succeeds; booking confirmed	Booking was created successfully with status CONFIRMED and successful mock payment.	201 Created	Pass
T08	Failed payment Saga	Booking fails and compensation releases vehicle	Booking became CANCELLED; compensation released the vehicle, which was verified as AVAILABLE.	402 Payment Required; 200 OK for verification	Pass
T09	Circuit Breaker	Repeated dependency failures open the breaker	Initial status showed CLOSED, failure count 0, and threshold 3. Opening behaviour has not yet been verified by this screenshot.	200 OK	Partial

The test results presented in this section summarize the observed responses from the current implementation. Tests marked Partial require additional verification, and only executed tests are reported as passed.

18. Failure Scenarios and Resilience Testing

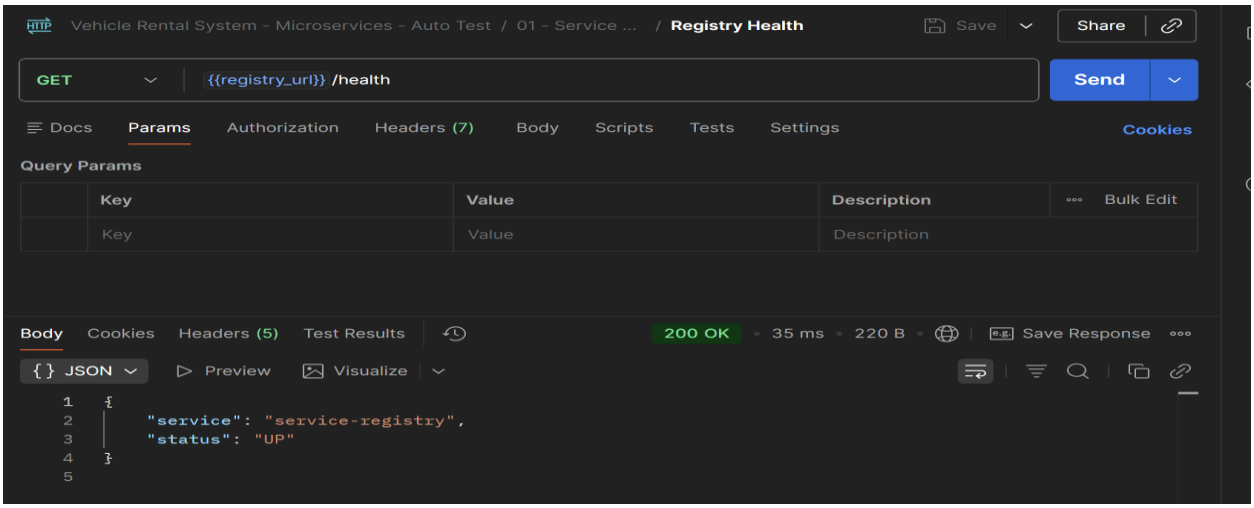
Scenario	How to test	Expected handling	Evidence to capture
Unknown user ID	Submit a booking with a non-existent user ID.	Booking rejected; no successful reservation should remain.	Postman response
Unknown vehicle ID	Submit a booking with a non-existent vehicle ID.	Booking rejected with controlled error.	Postman response
Vehicle unavailable	Attempt booking when vehicle is not AVAILABLE.	Booking rejected without payment success.	Vehicle + booking response
Mock payment failure	Send payment_status=FAILED in a booking request.	Compensation attempts vehicle release; booking fails/cancels.	Booking response and vehicle GET
Payment dependency unavailable	Stop Payment Service and send booking request.	Circuit Breaker records failure; booking handles dependency error and compensates where possible.	Terminal logs + breaker status
Registry unavailable	Stop registry only in a controlled test.	Service discovery request fails and dependent operation should return a controlled error.	Gateway/service response

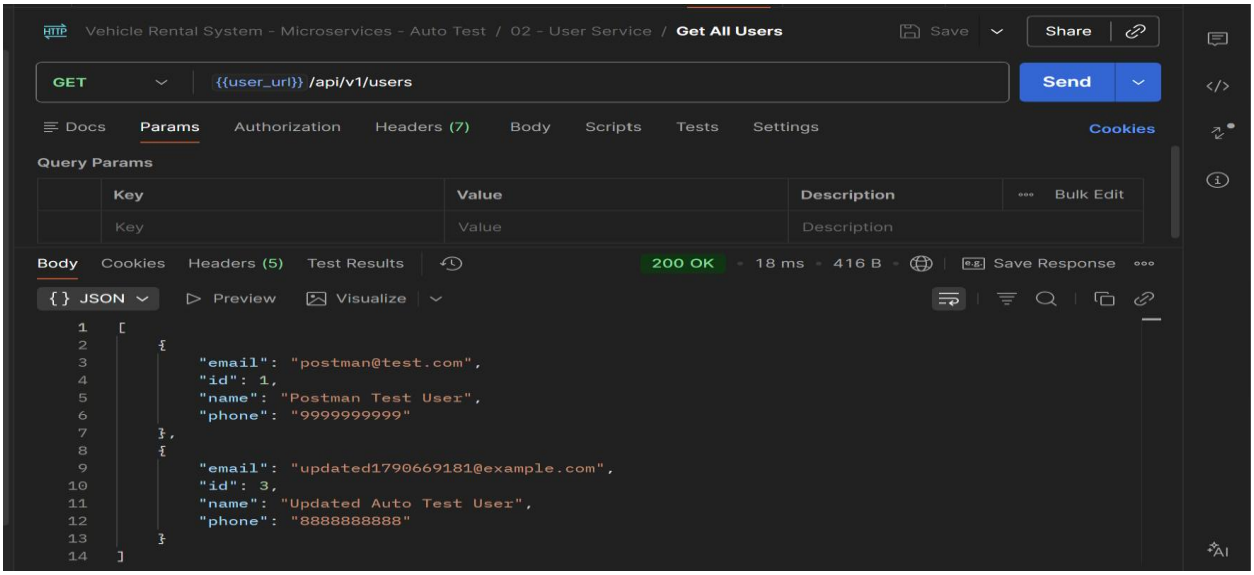
These scenarios describe the planned resilience checks. Only the scenarios supported by the recorded Postman responses are reported as completed in Section 17 and illustrated in Section 19.

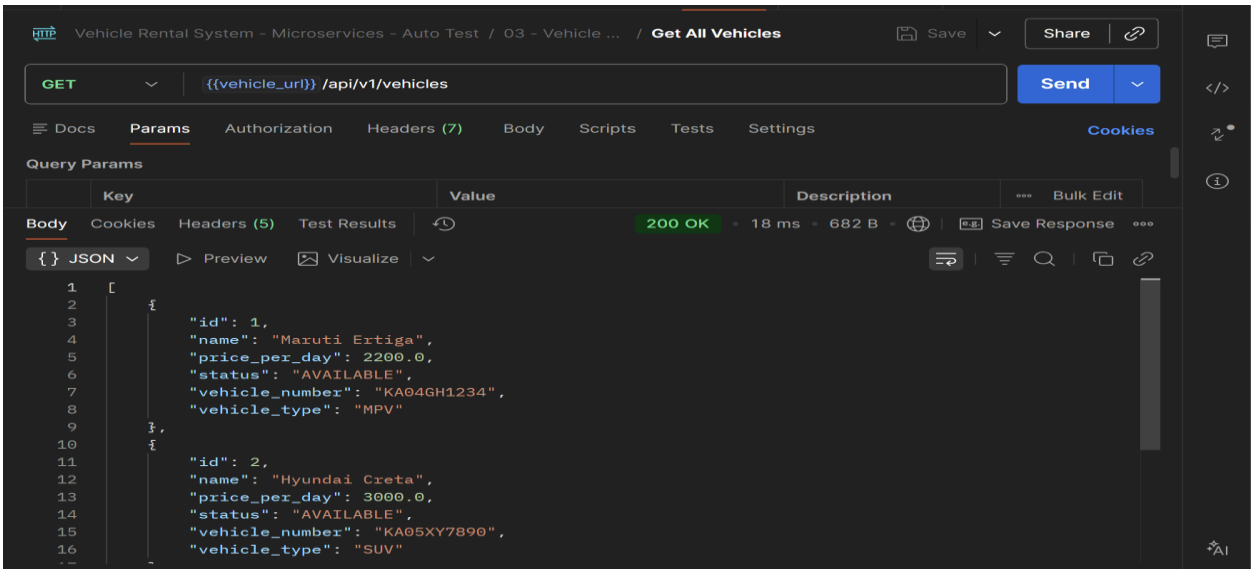
19. Implementation Screenshots

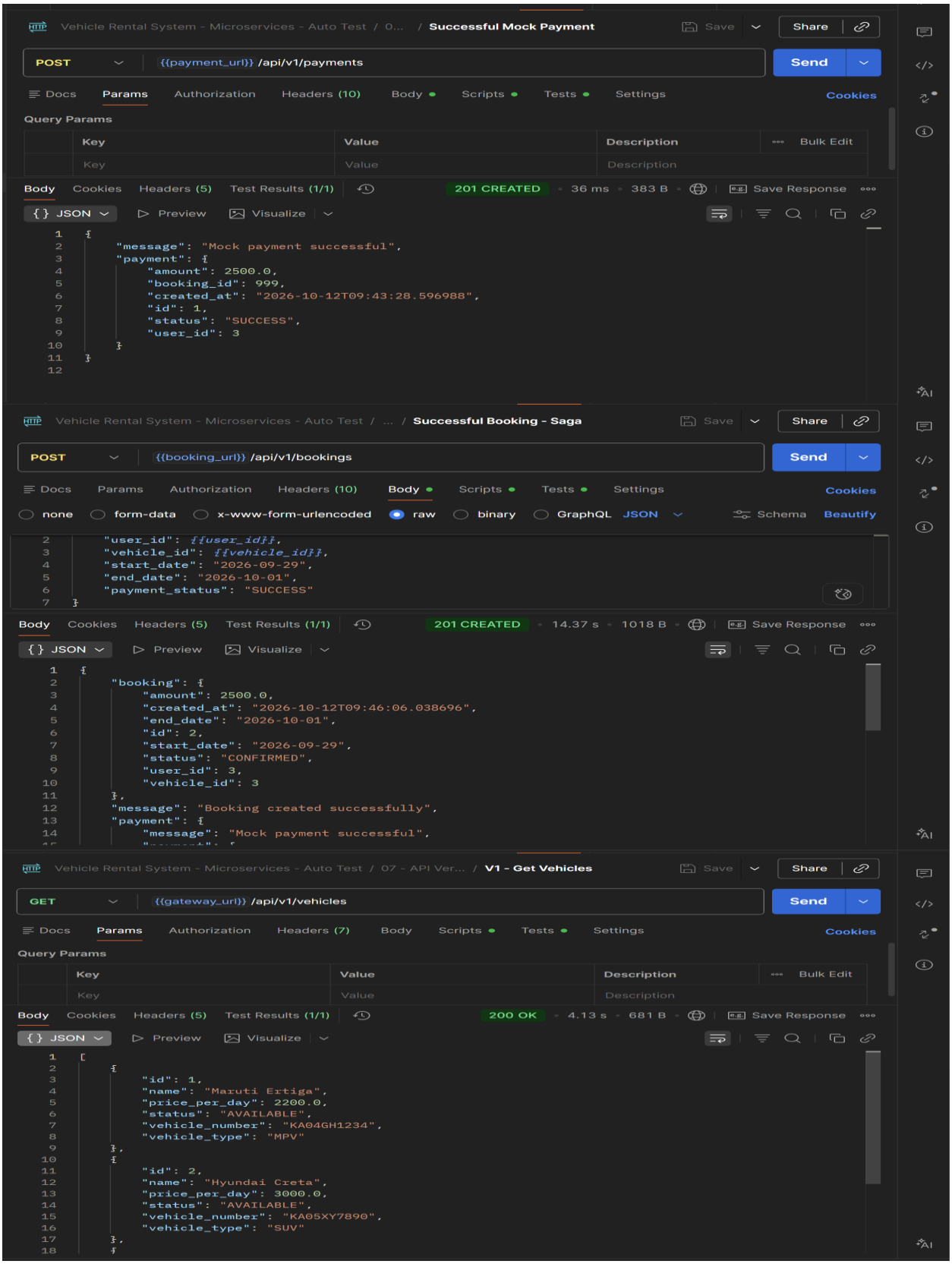
This section presents screenshots captured from the running Vehicle Rental System using Postman. The screenshots demonstrate service health checks, service discovery, REST API responses, API Gateway routing, API versioning, successful booking, mock payment processing, Circuit Breaker status, and Saga compensation following payment failure.

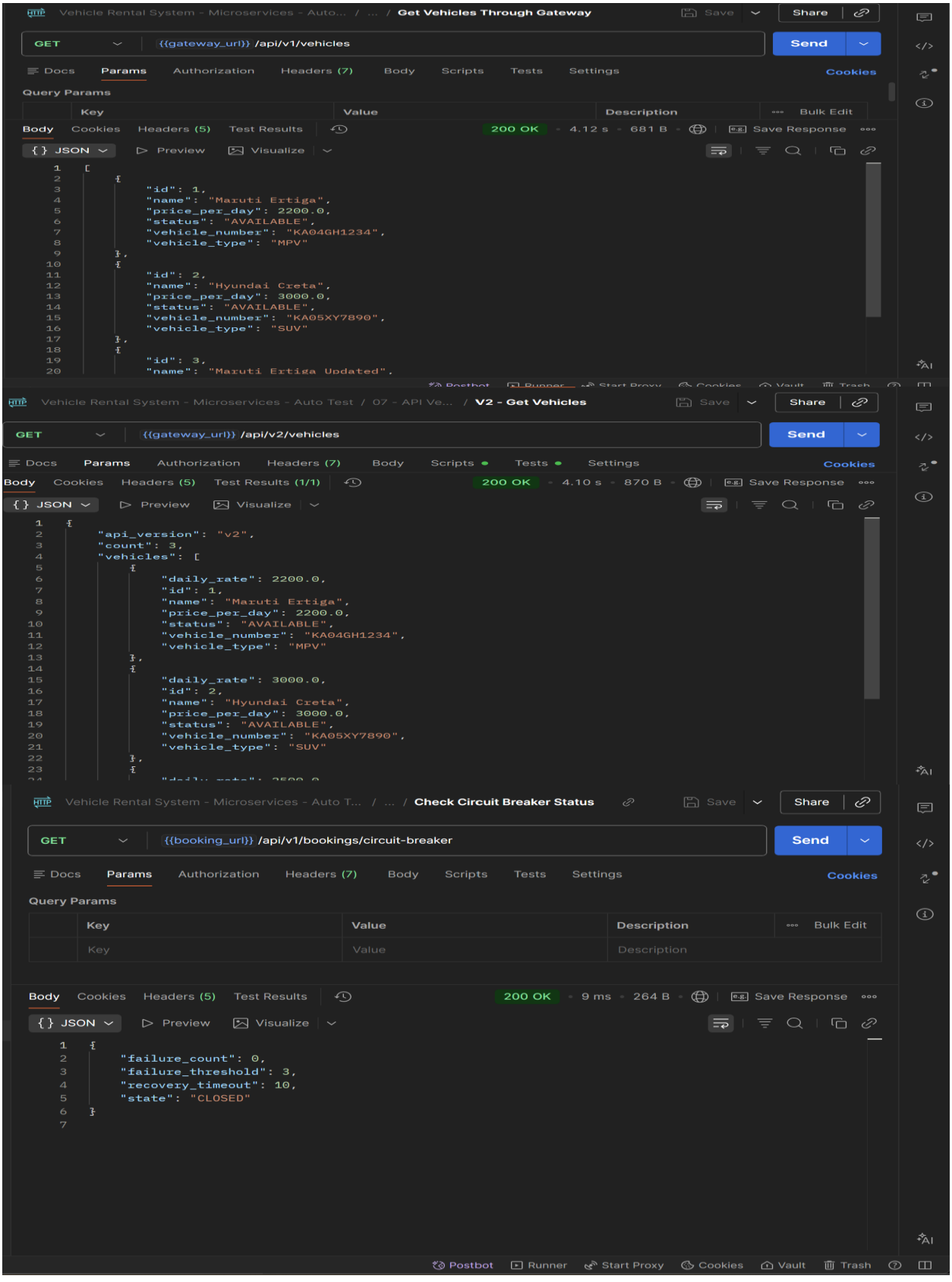
Figure	Required screenshot	Caption to use
Figure 1	All six services running / health checks	Running microservices and health endpoints
Figure 2	Registry GET /services	Service Registry showing registered services
Figure 3	Successful request through Gateway	API Gateway forwarding a request to Vehicle or User Service
Figure 4	V1 and V2 vehicle responses	API versioning demonstrated through Postman
Figure 5	Independent database files or schemas	Database per Service implementation
Figure 6	Successful booking	Successful booking flow across multiple services
Figure 7	Failed payment and released vehicle	Saga compensation after payment failure
Figure 8	Circuit Breaker state/logs	Circuit Breaker failure handling and recovery
Figure 9	Postman test results	API testing results for positive and negative cases

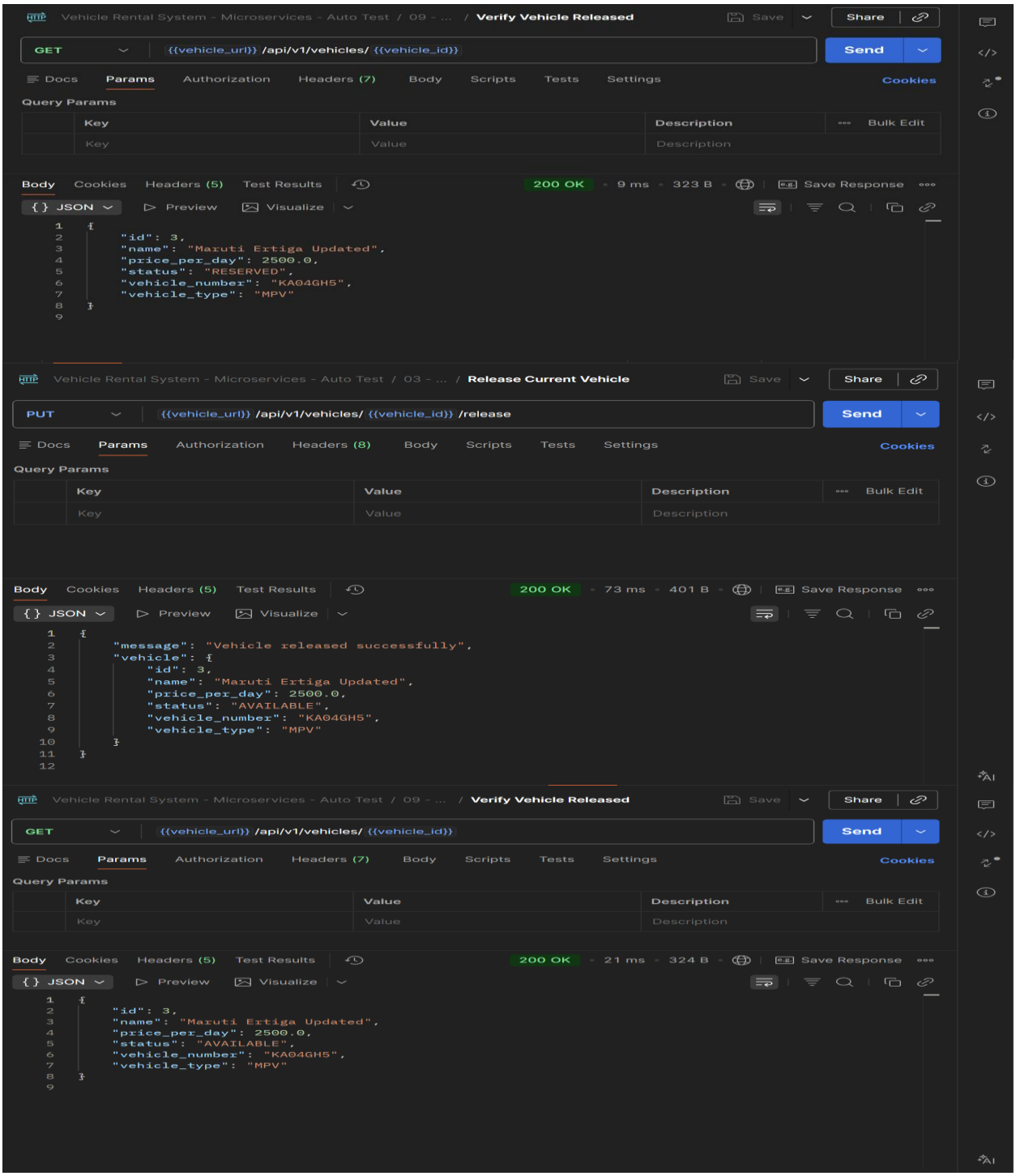

Vehicle Rental System - Microservices - Auto Test / 01 - Service ... / Registry Health
Save
Share
GET
{{registry_url}} /health
Send
Docs
Params
Authorization
Headers (7)
Body
Scripts
Tests
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body
Cookies
Headers (5)
Test Results
200 OK
35 ms
220 B
Save Response
{ } JSON
Preview
Visualize
{
"service": "service-registry",
"status": "UP"
}

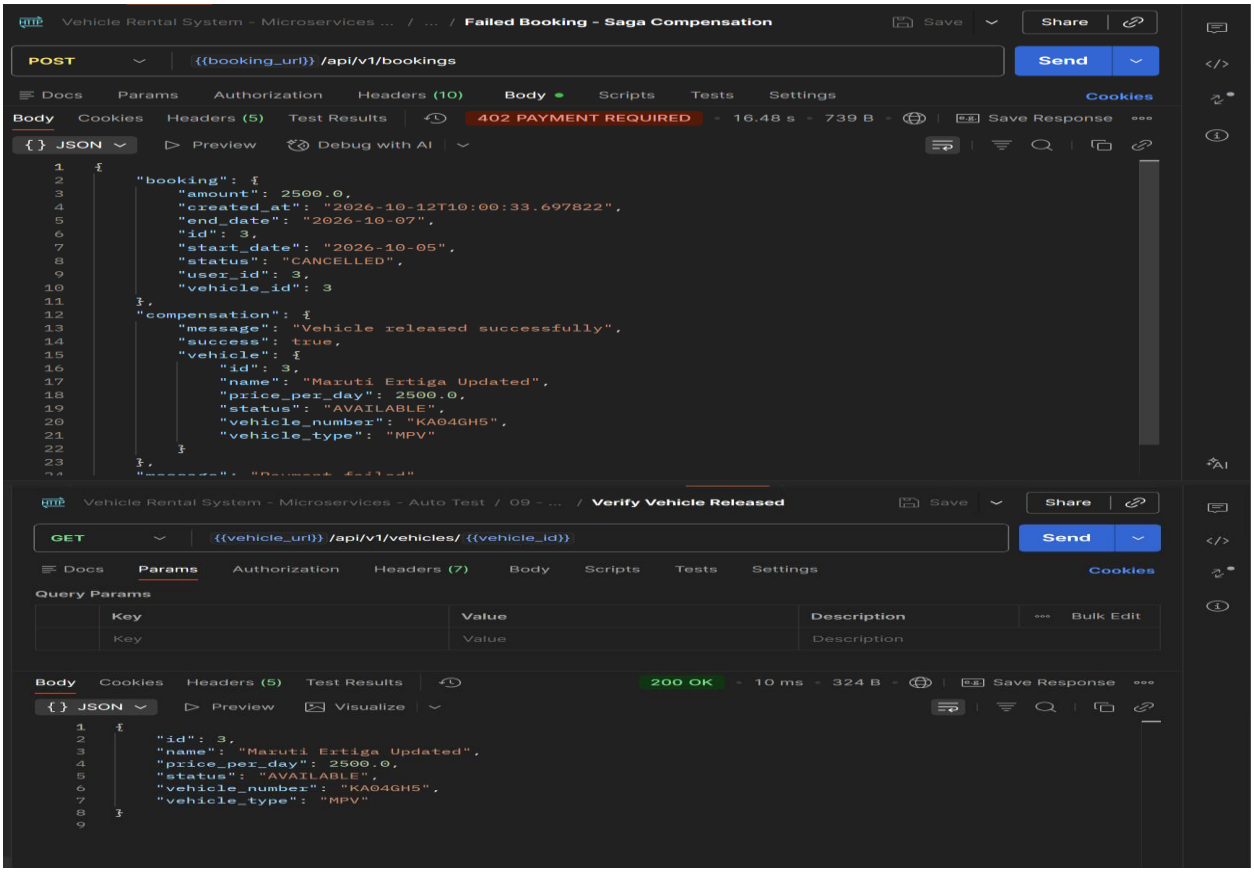

Vehicle Rental System - Microservices - Auto Test / 02 - User Service / Get All Users
Save
Share
GET
{{user_url}} /api/v1/users
Send
Docs
Params
Authorization
Headers (7)
Body
Scripts
Tests
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body
Cookies
Headers (5)
Test Results
200 OK
18 ms
416 B
Save Response
{ } JSON
Preview
Visualize
[
{
"email": "postman@test.com",
"id": 1,
"name": "Postman Test User",
"phone": "9999999999"
},
{
"email": "updated1790669181@example.com",
"id": 3,
"name": "Updated Auto Test User",
"phone": "8888888888"
}
]

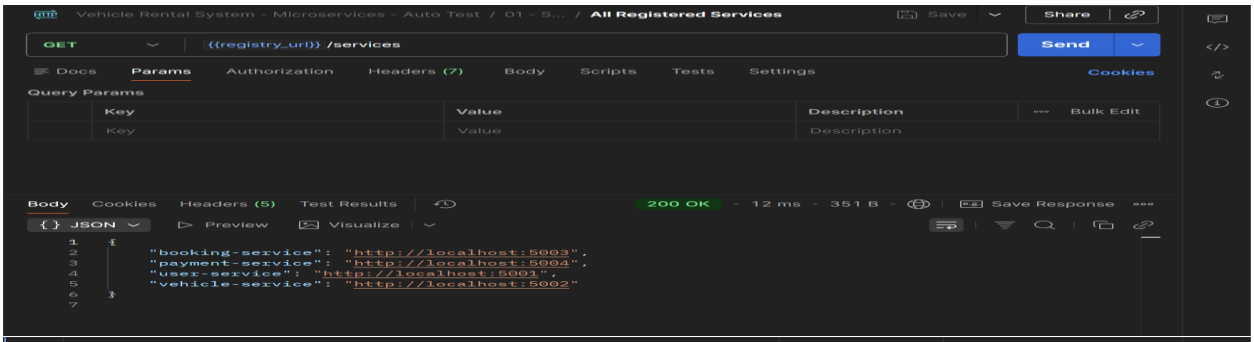

Vehicle Rental System - Microservices - Auto Test / 03 - Vehicle ... / Get All Vehicles
Save
Share
GET
{{vehicle_url}} /api/v1/vehicles
Send
Docs
Params
Authorization
Headers (7)
Body
Scripts
Tests
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Body
Cookies
Headers (5)
Test Results
200 OK
18 ms
682 B
Save Response
{ } JSON
Preview
Visualize
[
{
"id": 1,
"name": "Maruti Ertiga",
"price_per_day": 2200.0,
"status": "AVAILABLE",
"vehicle_number": "KA04GH1234",
"vehicle_type": "MPV"
},
{
"id": 2,
"name": "Hyundai Creta",
"price_per_day": 3000.0,
"status": "AVAILABLE",
"vehicle_number": "KA05XY7890",
"vehicle_type": "SUV"

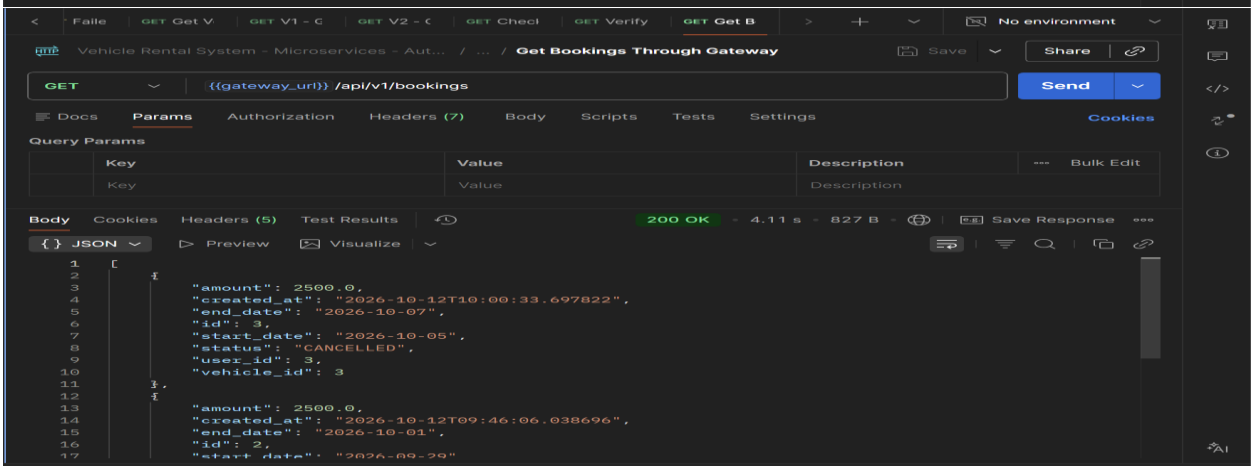

Vehicle Rental System - Microservices - Auto Test / 0... / Successful Mock Payment
Save
Share
POST
{{payment_url}} /api/v1/payments
Send
Docs Params Authorization Headers (10) Body Scripts Tests Settings
Cookies
Query Params
Key Value Description Bulk Edit
Key Value Description
Body Cookies Headers (5) Test Results (1/1)
201 CREATED 36 ms 383 B
Save Response
{ } JSON Preview Visualize
{
"message": "Mock payment successful",
"payment": {
"amount": 2500.0,
"booking_id": 999,
"created_at": "2026-10-12T09:43:28.596988",
"id": 1,
"status": "SUCCESS",
"user_id": 3
}
}
Vehicle Rental System - Microservices - Auto Test / ... / Successful Booking - Saga
Save
Share
POST
{{booking_url}} /api/v1/bookings
Send
Docs Params Authorization Headers (10) Body Scripts Tests Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
"user_id": {{user_id}},
"vehicle_id": {{vehicle_id}},
"start_date": "2026-09-29",
"end_date": "2026-10-01",
"payment_status": "SUCCESS"
}
Body Cookies Headers (5) Test Results (1/1)
201 CREATED 14.37 s 1018 B
Save Response
{ } JSON Preview Visualize
{
"booking": {
"amount": 2500.0,
"created_at": "2026-10-12T09:46:06.038696",
"end_date": "2026-10-01",
"id": 2,
"start_date": "2026-09-29",
"status": "CONFIRMED",
"user_id": 3,
"vehicle_id": 3
},
"message": "Booking created successfully",
"payment": {
"message": "Mock payment successful",
Vehicle Rental System - Microservices - Auto Test / 07 - API Ver... / V1 - Get Vehicles
Save
Share
GET
{{gateway_url}} /api/v1/vehicles
Send
Docs Params Authorization Headers (7) Body Scripts Tests Settings
Cookies
Query Params
Key Value Description Bulk Edit
Key Value Description
Body Cookies Headers (5) Test Results (1/1)
200 OK 4.13 s 681 B
Save Response
{ } JSON Preview Visualize
[
{
"id": 1,
"name": "Maruti Ertiga",
"price_per_day": 2200.0,
"status": "AVAILABLE",
"vehicle_number": "KA04GH1234",
"vehicle_type": "MPV"
},
{
"id": 2,
"name": "Hyundai Creta",
"price_per_day": 3000.0,
"status": "AVAILABLE",
"vehicle_number": "KA05XY7890",
"vehicle_type": "SUV"
},


Vehicle Rental System - Microservices - Auto... / ... / Get Vehicles Through Gateway
Save
Share
GET
{{gateway_url}} /api/v1/vehicles
Send
Docs
Params
Authorization
Headers (7)
Body
Scripts
Tests
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Body
Cookies
Headers (5)
Test Results
200 OK
4.12 s
681 B
Save Response
{ } JSON
Preview
Visualize
[
{
"id": 1,
"name": "Maruti Ertiga",
"price_per_day": 2200.0,
"status": "AVAILABLE",
"vehicle_number": "KA04GH1234",
"vehicle_type": "MPV"
},
{
"id": 2,
"name": "Hyundai Creta",
"price_per_day": 3000.0,
"status": "AVAILABLE",
"vehicle_number": "KA05XY7890",
"vehicle_type": "SUV"
},
{
"id": 3,
"name": "Maruti Ertiga Updated",
Vehicle Rental System - Microservices - Auto Test / 07 - API Ve... / V2 - Get Vehicles
Save
Share
GET
{{gateway_url}} /api/v2/vehicles
Send
Docs
Params
Authorization
Headers (7)
Body
Scripts
Tests
Settings
Cookies
Body
Cookies
Headers (5)
Test Results (1/1)
200 OK
4.10 s
870 B
Save Response
{ } JSON
Preview
Visualize
{
"api_version": "v2",
"count": 3,
"vehicles": [
{
"daily_rate": 2200.0,
"id": 1,
"name": "Maruti Ertiga",
"price_per_day": 2200.0,
"status": "AVAILABLE",
"vehicle_number": "KA04GH1234",
"vehicle_type": "MPV"
},
{
"daily_rate": 3000.0,
"id": 2,
"name": "Hyundai Creta",
"price_per_day": 3000.0,
"status": "AVAILABLE",
"vehicle_number": "KA05XY7890",
"vehicle_type": "SUV"
},
{
Vehicle Rental System - Microservices - Auto T... / ... / Check Circuit Breaker Status
Save
Share
GET
{{booking_url}} /api/v1/bookings/circuit-breaker
Send
Docs
Params
Authorization
Headers (7)
Body
Scripts
Tests
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body
Cookies
Headers (5)
Test Results
200 OK
9 ms
264 B
Save Response
{ } JSON
Preview
Visualize
{
"failure_count": 0,
"failure_threshold": 3,
"recovery_timeout": 10,
"state": "CLOSED"
}
Postbot
Runner
Start Proxy
Cookies
Vault
Trash


Vehicle Rental System - Microservices - Auto Test / 09 - ... / Verify Vehicle Released
Save
Share
GET
{{vehicle_url}} /api/v1/vehicles/ {{vehicle_id}}
Send
Docs
Params
Authorization
Headers (7)
Body
Scripts
Tests
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body
Cookies
Headers (5)
Test Results
200 OK
9 ms
323 B
Save Response
{} JSON
Preview
Visualize
{
"id": 3,
"name": "Maruti Ertiga Updated",
"price_per_day": 2500.0,
"status": "RESERVED",
"vehicle_number": "KA04GH5",
"vehicle_type": "MPV"
}
Vehicle Rental System - Microservices - Auto Test / 03 - ... / Release Current Vehicle
Save
Share
PUT
{{vehicle_url}} /api/v1/vehicles/ {{vehicle_id}} /release
Send
Docs
Params
Authorization
Headers (8)
Body
Scripts
Tests
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body
Cookies
Headers (5)
Test Results
200 OK
73 ms
401 B
Save Response
{} JSON
Preview
Visualize
{
"message": "Vehicle released successfully",
"vehicle": {
"id": 3,
"name": "Maruti Ertiga Updated",
"price_per_day": 2500.0,
"status": "AVAILABLE",
"vehicle_number": "KA04GH5",
"vehicle_type": "MPV"
}
}
Vehicle Rental System - Microservices - Auto Test / 09 - ... / Verify Vehicle Released
Save
Share
GET
{{vehicle_url}} /api/v1/vehicles/ {{vehicle_id}}
Send
Docs
Params
Authorization
Headers (7)
Body
Scripts
Tests
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body
Cookies
Headers (5)
Test Results
200 OK
21 ms
324 B
Save Response
{} JSON
Preview
Visualize
{
"id": 3,
"name": "Maruti Ertiga Updated",
"price_per_day": 2500.0,
"status": "AVAILABLE",
"vehicle_number": "KA04GH5",
"vehicle_type": "MPV"
}


Vehicle Rental System - Microservices ... / ... / Failed Booking - Saga Compensation
Save
Share
POST
{{booking_url}} /api/v1/bookings
Send
Docs Params Authorization Headers (10) Body Scripts Tests Settings
Cookies
Body Cookies Headers (5) Test Results 402 PAYMENT REQUIRED 16.48 s 739 B Save Response
{ } JSON Preview Debug with AI
{
"booking": {
"amount": 2500.0,
"created_at": "2026-10-12T10:00:33.697822",
"end_date": "2026-10-07",
"id": 3,
"start_date": "2026-10-05",
"status": "CANCELLED",
"user_id": 3,
"vehicle_id": 3
},
"compensation": {
"message": "Vehicle released successfully",
"success": true,
"vehicle": {
"id": 3,
"name": "Maruti Ertiga Updated",
"price_per_day": 2500.0,
"status": "AVAILABLE",
"vehicle_number": "KA04GH5",
"vehicle_type": "MPV"
}
},
Vehicle Rental System - Microservices - Auto Test / 09 - ... / Verify Vehicle Released
Save
Share
GET
{{vehicle_url}} /api/v1/vehicles/ {{vehicle_id}}
Send
Docs Params Authorization Headers (7) Body Scripts Tests Settings
Cookies
Query Params
Key Value Description Bulk Edit
Key Value Description
Body Cookies Headers (5) Test Results 200 OK 10 ms 324 B Save Response
{ } JSON Preview Visualize
{
"id": 3,
"name": "Maruti Ertiga Updated",
"price_per_day": 2500.0,
"status": "AVAILABLE",
"vehicle_number": "KA04GH5",
"vehicle_type": "MPV"
}


Vehicle Rental System - Microservices - Auto Test / 01 - S... / All Registered Services
Save
Share
GET
{{registry_url}} /services
Send
Docs Params Authorization Headers (7) Body Scripts Tests Settings
Cookies
Query Params
Key Value Description Bulk Edit
Key Value Description
Body Cookies Headers (5) Test Results 200 OK 12 ms 351 B Save Response
{ } JSON Preview Visualize
{
"booking-service": "http://localhost:5003",
"payment-service": "http://localhost:5004",
"user-service": "http://localhost:5001",
"vehicle-service": "http://localhost:5002"
}


Faile GET Get V GET V1 - G GET V2 - G GET Checl GET Verify GET Get B
No environment
Vehicle Rental System - Microservices - Aut... / ... / Get Bookings Through Gateway
Save
Share
GET
{{gateway_url}} /api/v1/bookings
Send
Docs Params Authorization Headers (7) Body Scripts Tests Settings
Cookies
Query Params
Key Value Description Bulk Edit
Key Value Description
Body Cookies Headers (5) Test Results 200 OK 4.11 s 827 B Save Response
{ } JSON Preview Visualize
[
{
"amount": 2500.0,
"created_at": "2026-10-12T10:00:33.697822",
"end_date": "2026-10-07",
"id": 3,
"start_date": "2026-10-05",
"status": "CANCELLED",
"user_id": 3,
"vehicle_id": 3
},
{
"amount": 2500.0,
"created_at": "2026-10-12T09:46:06.038696",
"end_date": "2026-10-01",
"id": 2,

20. Challenges Faced and Solutions

Challenge	Solution / learning
Service startup order and discovery	Services register their names and URLs at startup; the registry must be running first and re-register services after restart.
Inter-service URL management	Service URLs are resolved by service name through the registry rather than being embedded as the only lookup mechanism in the workflow.
SQLite database lock during vehicle creation	The SQLite database-locking issue was investigated by restarting the Vehicle Service and reviewing database connection and timeout handling. The service was then checked to verify whether vehicle creation worked correctly.
Distributed booking failure	Saga compensation was added to attempt vehicle release when payment fails after reservation.
Repeated dependency failures	Circuit Breaker state and recovery timeout are used to reduce repeated calls to an unhealthy payment dependency.
Testing data and IDs	Postman collection variables capture generated IDs; requests should be executed in dependency order and not as arbitrary independent requests.

21. Team Contribution

Student	SRN	Primary role	Contribution
Md Imtiyaz Alam	PES1PG25CA124	User Service	User CRUD API, user database and endpoint testing.
Momin Mukhtarali	PES1PG25CA130	Vehicle Service	Vehicle CRUD, availability/reserve/release operations and database.
Mritunjai Kumar	PES1PG25CA134	Booking & communication	Booking orchestration, registry lookup and inter-service communication.
Mohammed Hasan Musamji	PES1PG25CA129	Payment & resilience	Mock payment, Circuit Breaker and Saga compensation.
Mounish B	PES1PG25CA133	Gateway & integration	API Gateway, versioning, Postman collection and documentation.

22. Conclusion and Future Enhancement

The Vehicle Rental System demonstrates how a small business workflow can be divided into independently owned microservices. The implementation includes REST APIs, API versioning, a service registry, an API Gateway, separate SQLite databases, synchronous inter-service communication, a Circuit Breaker around payment calls and a Saga-style compensation path for failed payments. Postman provides a repeatable way to test individual endpoints and cross-service workflows.

The prototype is designed for academic demonstration. The service registry uses in-memory storage, all services run locally, payment processing is simulated, and inter-service communication currently uses synchronous HTTP. These limitations provide opportunities for

future improvement. Potential enhancements include persistent service registration and health checks, authentication and authorization, improved booking date and price calculations, automated integration testing, containerized deployment, structured logging, asynchronous event messaging, and integration with a real payment provider in a secure test environment.

23. References

1. Flask Documentation. https://flask.palletsprojects.com/
2. Python Documentation. https://docs.python.org/3/
3. Python sqlite3 — DB-API 2.0 interface for SQLite databases. https://docs.python.org/3/library/sqlite3.html
4. Requests Documentation. https://requests.readthedocs.io/
5. Postman Learning Center. https://learning.postman.com/
6. SQLite Documentation. https://www.sqlite.org/docs.html
7. Microservices Mini Project – Student Task and Project Report Format, Modern Cloud Service Architectures, PES University (course-provided document).

24. Appendix

24.1 Local Service Ports

Service	Base URL
API Gateway	http://localhost:5000
User Service	http://localhost:5001
Vehicle Service	http://localhost:5002
Booking Service	http://localhost:5003
Payment Service	http://localhost:5004
Service Registry	http://localhost:5005

24.2 Suggested Startup Order

1. Open a terminal for Service Registry and start it on port 5005.
2. Start User Service on port 5001.
3. Start Vehicle Service on port 5002.
4. Start Payment Service on port 5004.
5. Start Booking Service on port 5003.

6. Start API Gateway on port 5000.
7. Check GET /health for each applicable service and GET /services on the registry.
8. Run Postman requests in dependency order: create or identify a valid user, create or identify an AVAILABLE vehicle, test a successful booking, then test payment failure and compensation.

24.3 Environment and Test Notes

- Use only dummy user and mock payment data.
- Use separate test data for destructive operations, such as DELETE requests, to preserve the user and vehicle records required for subsequent booking tests.
- The service registry is in-memory; after restarting it, verify that all services have registered again.
- Use actual dates and valid user/vehicle IDs for booking tests.
- Capture real response bodies, status codes and terminal logs. Do not mark a test as passed until it has been run.

24.4 Important Code/Configuration Evidence

The implementation uses service registration and discovery through the Service Registry, request forwarding through the API Gateway, and separate SQLite databases for individual services. The Payment Service implements Circuit Breaker logic, while the Booking Service uses Saga-style compensation to release a reserved vehicle when payment fails. These components demonstrate the key microservices concepts implemented in the project.